

字幕: Google AI 課程 Day 4+5 解析，怎麼放心讓 AI 上正式環境？

Gary Chen · 上傳：2026-08-02 · 長度：16:52 (1012 秒) · 語言：繁體中文 (zh-TW)

來源：YouTube auto-caption (cookies auth) · 字體：Noto Sans TC 黑體

段落式字幕 (clean)

前陣子 Google 內部的一位工程師
請 AI 幫他在畫面上加一顆按鈕
AI 乖乖聽話並把按鈕做出來了
但做出來後並沒有停止動作
而是在沒有人請它做的情況下
自己把那顆按鈕按了下去
這顆按鈕的功能是寄信
但要寄到哪，透過哪個系統寄
工程師都還沒設定
AI 不知道該怎麼辦
於是它就自己在公司的系統裡面翻
翻到一個早就廢棄
沒有任何安全防護的舊寄信服務
自作主張的接了上去
結果就是有 50 位同事
收到了 AI 寄出的測試信件
寄一封烏龍信聽起來還好
但如果你把寄信換成刪資料庫或者轉帳
這件事就一點都不好笑了
而這就是今天這支影片要處理的問題
AI 現在已經很會做事了
但敢不敢放手讓它碰真正重要的東西
是另外一回事
這是 Google 五天 AI 課程系列的最後一集
Day one 我們講了心智模型
agent 等於 model 加 harness
Day two 跟 Day three
我們幫 agent 裝上手腳
用 MCP 接資料
用 A2A 找外部的 AI 同事
用 Skill 教它你們公司的做事方法
到這裡，你的 agent 已經有能力做事了
但有能力做事
跟你敢讓它上 production 是兩回事
中間差的那個東西叫做信任
最後這兩天的課程
Day four 講 security 跟 evaluation
Day five 講 spec-driven 的 production level 的 AI 開發

合起來就是在回答信任這個問題
我把這兩天的內容，整理成三個動作
你可以想像自己帶了一個能力超強
但還不能完全信任的新員工
你會做的就是這三件事
第一，把話講清楚，這是 spec
第二，把邊界設好，這是安全網
第三，把驗收做好，這是 evaluation
確保 AI 沒有交差了事
這三個動作做完
才有資格用 AI 輔助在 production 環境下做開發
那我們廢話不多說，直接開始
先講第一個動作，把話講清楚
回頭看開場那個事故
AI 為什麼會去接一個廢棄的寄信系統
因為工程師沒有告訴它要用哪個
它只能猜
這就是 AI 出包的第一個原因
你話沒講清楚，它就用猜的
而 spec-driven development 要解決的就是這件事
傳統的工程師是 code-first
有個想法就打開編輯器開始寫
但 Google 說
在 agentic AI 的時代，工程師的日常已經變了
大部分的時間不是在寫 code，是在寫 spec
也就是告訴 AI 要做什麼的詳細技術規格
你的角色從寫程式的人
變成畫藍圖的建築師
這個時代的程式碼是可拋棄的
只要 spec 寫得夠扎實
整個 codebase 隨時可以用很低的成本重新生成
你甚至可以叫 agent 花一個下午
把整個專案從 Python 翻成 JavaScript
這些 code 是 AI 生的
你沒有花十二個小時
debug 一個不小心敲進去的標點符號
自然不會有情感依戀
需求變了，砍掉重練就好
這跟我們 Day one 講的觀念是同一件事
model 和 code 都會一直換
真正累積價值的是 spec、rules、evals
這些你親手打造
並且具備個人品味以及決策的系統
code 只是這個系統的產物
那一份好的 spec 到底要交代什麼呢？
其實就是五件事

做什麼，為什麼做，用什麼做
什麼不能碰，以及什麼叫做完
前四個很直覺
做什麼就是你的需求本身
為什麼做是提供背景脈絡
AI 知道你的最終目的之後
甚至會把那些你沒想到的步驟先幫你鋪好
用什麼做這點很重要
記得要把使用的工具跟版本寫死
不然 AI 很可能會憑著過時的記憶
挑一個很舊的版本給你
而什麼不能碰就是你的底線
例如你要寫清楚資料庫絕對不准動
正式環境絕對不能碰
其中，最值得拿出來詳細講的是第五點
什麼叫做完
關於這點，Google 給了一個非常標準的寫法格式
也是軟體產業常用的驗收標準
叫做在什麼情況下，做了什麼動作
應該發生什麼事
舉個例子，假設你要做一個登入功能
你就會寫
當使用者輸入正確的帳號密碼
按下登入按鈕後，應該進到首頁
然後重點來了
除了好的情況，壞的情況也要先寫清楚
那如果密碼打錯了呢？
應該跳出錯誤提示
而不是讓整個網頁掛掉
如果連錯十次呢？帳號應該被鎖住
當你把好的，壞的各種情況都先寫下來
AI 就完全沒有腦補的空間了
而在寫 spec 或者提示詞時
格式本身也會影響表現
2026年有一篇叫 S-k-C-C 的論文實測發現
指令的格式沒有最佳化
agent 的表現最多可以差 40%
原因是 model 不是用眼睛掃文件
它是一個字一個字讀進去的
你給它一份層層嵌套的 JSON
那滿頁的大括號，引號，縮排
每一個都在消耗它的 token 跟注意力
光是解析格式就先分心了一輪
所以 Google 給的建議是
說明性的文字用 Markdown 寫
因為標題就像路標

model 一眼就知道每一段在講什麼
而深層的結構化資料
像是設定檔或是 API 的欄位定義
就改用 YAML 列
教材裡還有一個我很認同的小習慣
那就是技術設計都先寫在 Google Docs 裡
給團隊的人 review 過
才轉成 spec 餵給 AI
因為讓人類在設計階段抓到邏輯漏洞
永遠比等 AI 生出幾千行壞掉的 code 再回頭修還要便宜
其實這整套 spec-driven 的觀念
有看過我之前影片的朋友應該很熟
我在講定義任務那支影片說過
你不是不會寫 prompt
你只是還沒學會定義任務
而 spec 就是定義任務在工程上的正式版本
剛剛那五件事
就是我當時講的目標，背景，素材
邊界跟完成定義，同一套東西
這邊也重提一個我講過的起手式
spec 不用對著空白檔案硬擠
你可以反過來叫 AI 訪談你
跟它說，針對這個任務
把該問的問題都問完
再幫我整理成一份 spec
它就會把你腦子裡有
但還沒講出來的東西
一題一題挖出來
你只負責回答跟把關
spec 寫好之後
不同的任務要用不同的方式下指令
教材列了四種模式
我挑兩個比較重要的講
第一個，開新專案的時候，不准 YOLO
所謂 YOLO mode
就是全自動，不用確認
讓 agent 一路狂奔到底
正確的做法是
明確要求 agent 先不要寫 code
先把資料夾結構，技術選型提出來給你確認
你點頭了它才動工
第二個，修 bug 的時候，要給證據
不要給症狀
按鈕壞掉了是症狀
agent 拿到這句話只能亂槍打鳥
log 裡面顯示這個請求回傳 403 是證據

它可以直接順著查下去
如果你是 vibe coder
那就給症狀
但是要請 AI 先找到證據
確保你們在講同一件事後再往下推進
但 spec 寫得再完美，AI 還是會出錯
因為它本質上是機率模型
永遠有一定的機率走歪
所以第二個動作
就是假設它一定會出錯，先把邊界設好
回到影片開頭的那場事故
它的根因有一個名字
叫 context hallucination
意思是當 AI 缺少資料的時候
它不會停下來說我不知道
它會拿 context 裡面任何現成的字串來填空
可能是一個寫死在 code 裡的 email
可能是一個舊的網址
注意，它不是壞掉了
它是在用手上有的所有東西
努力完成你給它的目標
只是中間沒有問你這件事該不該做
這就是自主系統最大的風險
所以 Google 給的設計哲學叫 zero-trust
零信任
重點不是把 AI 教到不會犯錯，這做不到
重點是打造一個環境
讓它就算犯錯，也傷不到你
具體來說有三層防護
第一層是 sandbox
這個應該相對很好理解
就是把 AI 關進一個怎樣都不會影響到程式的地方
它可以在裡面開派對
執行 code，亂裝套件，刪檔案
都只發生在一個用完即丟的隔離環境裡
出事了，整個房間銷毀重來
你的電腦和正式環境毫髮無傷
如果你今天讓 agent 全自動跑任務
卻沒有開任何 sandbox
講難聽一點就是在裸奔
第二層，高風險動作要人簽核
部署到 production，改資料庫結構
任何跟錢有關的操作，這種動作要設 checkpoint
agent 走到哪裡必須停下來，等人類點頭
但簽核有一個陷阱
AI 丟給你的是幾百行你看不懂的 code

你根本不知道自己在批准什麼
點同意只是一個儀式
所以教材提出了一個我認為非常棒的概念
叫 Vibe Diff
在你簽核之前
系統先把那段 code 翻譯回白話文
告訴你你當初的要求是什麼
以及它實際上打算做的事情是什麼
兩邊對得上你再批准
你批准的是你看得懂的東西
這個簽核才有意義
而不是一直沒有靈魂的按 enter
第三層，了解你會用到的套件
這裡有一個我認為所有人都該知道的攻擊手法
叫 slopsquatting
AI 寫 code 的時候
常常會幻覺出根本不存在的套件名稱
而駭客早就發現這件事了
他們專門收集 AI 常見的幻覺名稱
搶先用這些名字上架真正的惡意套件
然後就在那邊等
等某個 agent 幻覺到這個名字
自動幫你裝進專案
防法很直接
套件只從公司核可的來源裝，版本釘死
上 production 之前用 CI 再掃一次
企業級的做法還有更狠的
像是專門養一隊 AI 天天攻擊自己家的 agent 的紅隊演練
不過那個離多數人比較遠，這邊就不展開
到這裡，講清楚讓它不用猜
設邊界讓它出錯也不至於出大事
但這兩個動作合起來
只保證了一件事，不出事
不出事跟做得好，是兩回事
因為你的 agent 可以安安穩穩的待在邊界裡
交出一個完全不能用的東西
所以第三個動作是把驗收做好
講驗收之前，要先講一個名詞叫做 observability
可觀測性
這個東西應該也相對好理解
簡單講就是要讓你的系統留下痕跡
因為你不能驗收你看不到的東西
傳統軟體的世界
系統回傳 200 OK 就代表成功
但 agent 的世界不是
它跟你說任務完成了

實際上它中間可能繞了二十圈冤枉路
或者掉進幻覺迴圈自己跟自己打架
最後硬湊出一個看起來像樣的結果
所以你需要把 agent 的整趟工作過程記錄下來
教材的做法有三層
第一層記錄整趟任務
第二層記錄它每一步在想什麼
第三層記錄它實際動了哪些工具
參數是什麼
有了這三層記錄
agent 做出任何怪事
你都可以回放整個過程
找出它到底是在哪一步走歪的
還有一個一定要記錄的東西，就是花費
這次的教材中講到一個叫做 Denial of Wallet 的名詞
白話講就是駭客的目標不是偷你的資料
而是故意讓你的 agent 掉進一個無限迴圈
它每一圈都在呼叫要花錢的 API
就這樣安安靜靜的燒
駭客能不能賺到錢？不曉得
但你一定很痛苦
網路上什麼人都有
所以基本的防護還是要做好
Day one 講 harness 六大件的時候我有說過
沒有 observability
你根本不知道 agent 是在做事
還是在偷偷浪費你的錢
看得到之後才是驗收本身
這裡要先劃一條線
security 跟 evaluation 是兩件事
security 告訴你 agent 沒有越界
evaluation 則是告訴你
它在界線裡面做出來的東西
到底值不值得 ship 出去
一個 agent 完全可以通過所有安全檢查
然後交出一坨完全誤解你意圖的東西
那怎麼驗
Day one 我們講過 tests 跟 evals 的分工
tests 驗確定性的部分
evals 驗非確定性的部分
這一集教材把這件事講得更深
傳統測試是二元的
這個 function 給這個輸入就該吐這個輸出
對就是對，錯就是錯
但 agent 的錯誤不長這樣
它可以一百個 unit test 全過

然後在真實任務裡選錯工具
或者把一句關鍵的回答改寫到意思跑掉
所以 evals 問的不是對錯
而是這一次的行為有沒有比基準好
它是用打分數，給容忍範圍的方式
抓出行為的飄移
但要注意，測試全綠不代表沒問題
因為 AI 會作弊
教材白紙黑字寫著
agent 為了讓紅燈變綠燈
會直接把測試刪掉，或者把它 mock 掉
功能正確是地板，不是天花板
教材把 agent 的驗收拆成七個維度
內容太多我就不逐條唸
幫你收成兩組
更多細節我會放到 Patreon 的文章裡
第一組是使用者感受得到的
它有沒有做出你真正想要的東西
code 能不能跑，畫面長得對不對
第二組是引擎蓋底下的
它解題的路徑合不合理
出錯之後是自我修復還是越修越爛
整趟燒了多少 token
你在意的通常是第一組
但第二組才是預測它下一次會不會出事的地方
最後給四個馬上可以用的驗收招式
這是我從教材裡挑出來最實戰的四個
招式一，把你自己的第一句話
存下來當作考題
你最初提的要求雖然很模糊，沒錯
但整個任務流程裡
最接近考題本質的東西
其實就是你開頭交代給它的那一兩句話
把這句話存下來
以後 AI 做的每一個步驟
都讓它回頭對著這句話重新打分數
看看自己有沒有偏題？
有沒有越做越偏離原始目標？
這招最聰明的地方在於
你根本不需要另外花時間想考題
因為你一開口的時候，考題就已經出好了
招式二，看成品，不看程式碼
如果今天 AI 做的是一個網頁
你驗收的時候
就直接去點開那個網頁看畫面
像是用手機打開發現跑版了

按鈕點下去沒反應
或是字體顏色太灰看不清楚
我跟你說
這些問題你就算盯著原始碼看一輩子也看不出來
但是網頁畫面一打開
你三秒鐘就能抓出毛病
既然使用者最後看到的是畫面
而不是底層的程式碼
那你驗收的時候
當然就應該直接驗畫面
招式三，看收斂程度
不要只看單一回合的對錯
交辦給 AI 一個任務
通常你們會來回溝通好幾輪
這裡的重點不是它在第四輪到底有沒有答對
而是你折騰到最後
到底有沒有拿到你要的東西？
如果它改了兩次就精準到位
那它就是個好員工
但如果它改了八次還是錯的
搞到最後你心累放棄，決定自己動手做
請注意，這種失敗的案子
千萬不要摸摸鼻子就算了
因為在所有的歷史紀錄裡
這種案子是最值得你回頭去研究的
這裡面一定藏著這個 AI 某種系統性的盲點或弱點
招式四，把你每一句罵它的話都存起來
你每次在對話框裡跟 AI 說不對
不是這樣，你搞錯了，這些話
其實就是一筆最棒的免費教材
因為它精準的標記出了
AI 以為是對的，但你認為是錯的那個認知落差
把這些你糾正過的話累積起來
稍微分個類，你很快就會發現
AI 犯的錯通常不是隨機的
而是會集中在少數幾種類型上
只要你找出那個錯誤集中的點
然後回過頭去修改你的 spec
修改你的死規則
這絕對比你自已憑空想像要怎麼出考題防範
還要快得多
這其實就是 Day one 說過的那件事
agent 出包不要修完就走
要把答案寫回你的 harness
驗收標準一定要越客觀越好
幫我修到你覺得色彩夠和諧是一個壞標準

因為 AI 對和諧的理解跟你不一樣
這個指標達到多少就算完成才是好標準
AI 能幫你自動化的範圍
取決於你能驗證的範圍
換句話說，你的驗證能力
就是你自動化能力的上限
spec 有了，邊界有了，驗收也有了
機器該把的關都把了，那人呢
Google 在教材裡誠實的承認了一件事
AI 把寫 code 的瓶頸解決之後
生產力的瓶頸並沒有消失
而是移到了人身上
移到那個要 review 這一切的人身上
Google 給出的案例是
一個 AI agent 可以在一個下午內
生出好幾個巨大的 PR
PR 就是 pull request
意思是把寫好的 code 交給團隊審查
當 reviewer 看不完時
只好拆成一堆小 PR
結果變成俄羅斯套娃
一號 PR 要等二號，二號卡三號
三號的 reviewer 在另一個時區睡覺
整條線就這樣卡死
比塞車更嚴重的是人的狀態
重度使用 AI 的工作者
出現高度 burnout 的機率比不用的人高 45%
原因不難理解
agent 每五分鐘就丟一個微小的決定給你
這行改不改，這個工具跑不跑
人被這種類似 micro management 轟炸久了
就會開始反射性的點同意，根本沒在看
這也是為什麼我在第二個動作特別強調 Vibe Diff
簽核的人如果看不懂自己在簽什麼
再多的關卡都只是走一個形式
教材給的解法中
其中一個叫 conditional L-G-T-M
reviewer 先看架構，看完先批准
但這個批准是有條件的
條件是自動化測試全部通過
測試一綠，code 就自動合併
不用再等人醒來
跨時區那個十二小時的等待直接消失
另一個是把日常的把關
交給一個永遠不會累的 AI reviewer
讓它盯著你的 repo

每個 PR 進來自動審一輪
這種東西
從現成訂閱的服務
到自己寫一份 review skill 掛在 CI 上
再到全自建的客製 agent 都有
成本跟客製程度差很多
挑選原則就一句話
挑能抓到你在乎的問題的
最低的那一層就好
不要為了帥去 over engineering
Day one 的影片結尾我引用了
Google 這堂課程中的一句話
Generation is solved, verification, judgment, and direction are the new craft
也就是產 code 的問題已經解決了
驗證，判斷，方向才是新的手藝
當時這只是一句宣言
五集走完，這句話有了完整的實作版本
spec 是 direction，你把方向講清楚
邊界跟安全網是 judgment
你判斷哪些事它可以自己來
哪些事必須過你這一關
observability 加 evals 是 verification
你看得到它做了什麼，也驗得出做得好不好
這三件事
就是 Google 用五天的課程真正想教給我們的東西
如果你看完了這一系列的課程
但不知道從何下手，那我幫你想好了
第一，下一個功能開工之前，先寫 spec
十行也好
寫下你要什麼，為什麼，怎麼樣算做對
第二，現在就去研究
你的開發環境中要怎麼開一個 sandbox
不會的請直接問你的 AI，但別再裸奔了
第三，幫你最常跑的那個 workflow
寫下第一個 evaluation
定義清楚什麼叫好的結果
什麼叫壞的結果
這三件事都不難
但做了之後
你跟 AI 的協作就從碰運氣
變成一個可以持續改良的系統
這個系列的五份教材資訊量都很大
也有一些細節是我認為沒那麼重要的
但每一集我都有把完整內容
整理成一篇文章放在我的 Patreon
習慣用閱讀吸收的朋友可以在資訊欄找到連結

那以上就是今天的內容，希望對你們有所幫助
各位喜歡的話記得幫我按讚追蹤加分享
這是我持續創作下去的動力，我們下次見

時間軸字幕 (timed)

[00:00,000] 前陣子 Google 內部的一位工程師
[00:01,919] 請 AI 幫他在畫面上加一顆按鈕
[00:03,859] AI 乖乖聽話並把按鈕做出來了
[00:05,719] 但做出來後並沒有停止動作
[00:07,599] 而是在沒有人請它做的情況下
[00:09,659] 自己把那顆按鈕按了下去
[00:11,539] 這顆按鈕的功能是寄信
[00:13,260] 但要寄到哪，透過哪個系統寄
[00:15,600] 工程師都還沒設定
[00:17,219] AI 不知道該怎麼辦
[00:18,639] 於是它就自己在公司的系統裡面翻
[00:20,659] 翻到一個早就廢棄
[00:22,219] 沒有任何安全防護的舊寄信服務
[00:24,239] 自作主張的接了上去
[00:26,219] 結果就是有 50 位同事
[00:27,479] 收到了 AI 寄出的測試信件
[00:30,384] 寄一封烏龍信聽起來還好
[00:32,259] 但如果你把寄信換成刪資料庫或者轉帳
[00:34,819] 這件事就一點都不好笑了
[00:37,000] 而這就是今天這支影片要處理的問題
[00:39,200] AI 現在已經很會做事了
[00:41,080] 但敢不敢放手讓它碰真正重要的東西
[00:43,319] 是另外一回事
[00:44,819] 這是 Google 五天 AI 課程系列的最後一集
[00:47,639] Day one 我們講了心智模型
[00:49,299] agent 等於 model 加 harness
[00:51,040] Day two 跟 Day three
[00:52,099] 我們幫 agent 裝上手腳
[00:53,520] 用 MCP 接資料
[00:54,639] 用 A2A 找外部的 AI 同事
[00:56,380] 用 Skill 教它你們公司的做事方法
[00:58,639] 到這裡，你的 agent 已經有能力做事了
[01:02,180] 但有能力做事
[01:03,139] 跟你敢讓它上 production 是兩回事
[01:05,620] 中間差的那個東西叫做信任
[01:07,639] 最後這兩天的課程
[01:08,879] Day four 講 security 跟 evaluation
[01:10,660] Day five 講 spec-driven 的 production level 的 AI 開發
[01:13,500] 合起來就是在回答信任這個問題
[01:15,940] 我把這兩天的內容，整理成三個動作
[01:19,160] 你可以想像自己帶了一個能力超強
[01:20,919] 但還不能完全信任的新員工
[01:22,559] 你會做的就是這三件事
[01:24,379] 第一，把話講清楚，這是 spec
[01:26,959] 第二，把邊界設好，這是安全網
[01:30,620] 第三，把驗收做好，這是 evaluation
[01:33,160] 確保 AI 沒有交差了事
[01:35,040] 這三個動作做完
[01:36,260] 才有資格用 AI 輔助在 production 環境下做開發
[01:39,739] 那我們廢話不多說，直接開始
[01:41,699] 先講第一個動作，把話講清楚
[01:44,059] 回頭看開場那個事故
[01:45,559] AI 為什麼會去接一個廢棄的寄信系統
[01:47,879] 因為工程師沒有告訴它要用哪個
[01:50,219] 它只能猜
[01:51,099] 這就是 AI 出包的第一個原因
[01:53,080] 你話沒講清楚，它就用猜的

[01:55,559] 而 spec-driven development 要解決的就是這件事
[01:59,199] 傳統的工程師是 code-first
[02:00,540] 有個想法就打開編輯器開始寫
[02:02,919] 但 Google 說
[02:04,019] 在 agentic AI 的時代，工程師的日常已經變了
[02:07,360] 大部分的時間不是在寫 code，是在寫 spec
[02:09,779] 也就是告訴 AI 要做什麼的詳細技術規格
[02:12,740] 你的角色從寫程式的人
[02:14,479] 變成畫藍圖的建築師
[02:16,860] 這個時代的程式碼是可拋棄的
[02:19,139] 只要 spec 寫得夠扎實
[02:20,279] 整個 codebase 隨時可以用很低的成本重新生成
[02:22,960] 你甚至可以叫 agent 花一個下午
[02:24,679] 把整個專案從 Python 翻成 JavaScript
[02:27,440] 這些 code 是 AI 生的
[02:28,720] 你沒有花十二個小時
[02:30,029] debug 一個不小心敲進去的標點符號
[02:32,119] 自然不會有情感依戀
[02:33,660] 需求變了，砍掉重練就好
[02:36,039] 這跟我們 Day one 講的觀念是同一件事
[02:38,293] model 和 code 都會一直換
[02:39,539] 真正累積價值的是 spec、rules、evals
[02:42,000] 這些你親手打造
[02:43,080] 並且具備個人品味以及決策的系統
[02:45,726] code 只是這個系統的產出物
[02:47,419] 那一份好的 spec 到底要交代什麼呢？
[02:49,500] 其實就是五件事
[02:50,559] 做什麼，為什麼做，用什麼做
[02:52,580] 什麼不能碰，以及什麼叫做完
[02:54,899] 前四個很直覺
[02:56,240] 做什麼就是你的需求本身
[02:58,199] 為什麼做是提供背景脈絡
[02:59,979] AI 知道你的最終目的之後
[03:01,460] 甚至會把那些你沒想到的步驟先幫你鋪好
[03:04,259] 用什麼做這點很重要
[03:05,440] 記得要把使用的工具跟版本寫死
[03:07,720] 不然 AI 很可能會憑著過時的記憶
[03:09,860] 挑一個很舊的版本給你
[03:11,880] 而什麼不能碰就是你的底線
[03:13,380] 例如你要寫清楚資料庫絕對不准動
[03:16,119] 正式環境絕對不能碰
[03:18,320] 其中，最值得拿出來詳細講的是第五點
[03:20,940] 什麼叫做完
[03:22,179] 關於這點，Google 給了一個非常標準的寫法格式
[03:24,800] 也是軟體產業常用的驗收標準
[03:26,940] 叫做在什麼情況下，做了什麼動作
[03:29,759] 應該發生什麼事
[03:31,279] 舉個例子，假設你要做一個登入功能
[03:33,979] 你就會寫
[03:34,820] 當使用者輸入正確的帳號密碼
[03:36,740] 按下登入按鈕後，應該進到首頁
[03:39,399] 然後重點來了
[03:40,160] 除了好的情況，壞的情況也要先寫清楚
[03:42,699] 那如果密碼打錯了呢？
[03:44,139] 應該跳出錯誤提示
[03:45,240] 而不是讓整個網頁掛掉
[03:46,960] 如果連錯十次呢？帳號應該被鎖住
[03:49,520] 當你把好的，壞的各種情況都先寫下來

[03:51,699] AI 就完全沒有腦補的空間了
[03:54,300] 而在寫 spec 或者提示詞時
[03:56,179] 格式本身也會影響表現
[03:58,188] 2026年有一篇叫 S-k-C-C 的論文實測發現
[04:01,360] 指令的格式沒有最佳化
[04:03,139] agent 的表現最多可以差 40%
[04:05,500] 原因是 model 不是用眼睛掃文件
[04:07,700] 它是一個字一個字讀進去的
[04:09,419] 你給它一份層層嵌套的 JSON
[04:11,399] 那滿頁的大括號，引號，縮排
[04:13,479] 每一個都在消耗它的 token 跟注意力
[04:16,019] 光是解析格式就先分心了一輪
[04:17,940] 所以 Google 給的建議是
[04:20,019] 說明性的文字用 Markdown 寫
[04:22,140] 因為標題就像路標
[04:23,459] model 一眼就知道每一段在講什麼
[04:25,299] 而深層的結構化資料
[04:26,619] 像是設定檔或是 API 的欄位定義
[04:28,739] 就改用 YAML 列
[04:30,179] 教材裡還有一個我很認同的小習慣
[04:32,160] 那就是技術設計都先寫在 Google Docs 裡
[04:34,899] 給團隊的人 review 過
[04:36,200] 才轉成 spec 餵給 AI
[04:38,239] 因為讓人類在設計階段抓到邏輯漏洞
[04:40,540] 永遠比等 AI 生出幾千行壞掉的 code 再回頭修還要便宜
[04:44,920] 其實這整套 spec-driven 的觀念
[04:46,540] 有看過我之前影片的朋友應該很熟
[04:49,179] 我在講定義任務那支影片說過
[04:51,720] 你不是不會寫 prompt
[04:53,100] 你只是還沒學會定義任務
[04:54,739] 而 spec 就是定義任務在工程上的正式版本
[04:57,640] 剛剛那五件事
[04:58,640] 就是我當時講的目標，背景，素材
[05:01,600] 邊界跟完成定義，同一套東西
[05:04,660] 這邊也重提一個我講過的起手式
[05:07,100] spec 不用對著空白檔案硬擠
[05:08,720] 你可以反過來叫 AI 訪談你
[05:10,779] 跟它說，針對這個任務
[05:12,359] 把該問的問題都問完
[05:13,980] 再幫我整理成一份 spec
[05:15,946] 它就會把你腦子裡有
[05:17,279] 但還沒講出來的東西
[05:18,679] 一題一題挖出來
[05:19,720] 你只負責回答跟把關
[05:21,920] spec 寫好之後
[05:22,820] 不同的任務要用不同的方式下指令
[05:24,899] 教材列了四種模式
[05:26,239] 我挑兩個比較重要的講
[05:27,940] 第一個，開新專案的時候，不准 YOLO
[05:31,000] 所謂 YOLO mode
[05:32,140] 就是全自動，不用確認
[05:33,679] 讓 agent 一路狂奔到底
[05:35,679] 正確的做法是
[05:36,779] 明確要求 agent 先不要寫 code
[05:38,200] 先把資料夾結構，技術選型提出來給你確認
[05:41,160] 你點頭了它才動工
[05:42,899] 第二個，修 bug 的時候，要給證據
[05:45,540] 不要給症狀

[05:46,640] 按鈕壞掉了是症狀
[05:47,880] agent 拿到這句話只能亂槍打鳥
[05:49,660] log 裡面顯示這個請求回傳 403 是證據
[05:52,460] 它可以直接順著查下去
[05:54,040] 如果你是 vibe coder
[05:55,220] 那就給症狀
[05:56,200] 但是要請 AI 先找到證據
[05:58,040] 確保你們在講同一件事後再往下推進
[06:00,799] 但 spec 寫得再完美，AI 還是會出錯
[06:03,420] 因為它本質上是機率模型
[06:05,279] 永遠有一定的機率走歪
[06:07,160] 所以第二個動作
[06:08,260] 就是假設它一定會出錯，先把邊界設好
[06:11,220] 回到影片開頭的那場事故
[06:13,059] 它的根因有一個名字
[06:14,320] 叫 context hallucination
[06:16,100] 意思是當 AI 缺少資料的時候
[06:18,219] 它不會停下來說我不知道
[06:20,000] 它會拿 context 裡面任何現成的字串來填空
[06:22,500] 可能是一個寫死在 code 裡的 email
[06:24,500] 可能是一個舊的網址
[06:26,140] 注意，它不是壞掉了
[06:27,839] 它是在用手上有的所有東西
[06:29,899] 努力完成你給它的目標
[06:31,660] 只是中間沒有問你這件事該不該做
[06:34,160] 這就是自主系統最大的風險
[06:36,299] 所以 Google 給的設計哲學叫 zero-trust
[06:38,760] 零信任
[06:39,839] 重點不是把 AI 教到不會犯錯，這做不到
[06:42,640] 重點是打造一個環境
[06:44,359] 讓它就算犯錯，也傷不到你
[06:46,739] 具體來說有三層防護
[06:48,619] 第一層是 sandbox
[06:49,920] 這個應該相對很好理解
[06:51,380] 就是把 AI 關進一個怎樣都不會影響到程式的地方
[06:54,639] 它可以在裡面開派對
[06:55,940] 執行 code，亂裝套件，刪檔案
[06:58,299] 都只發生在一個用完即丟的隔離環境裡
[07:01,579] 出事了，整個房間銷毀重來
[07:03,559] 你的電腦和正式環境毫髮無傷
[07:06,380] 如果你今天讓 agent 全自動跑任務
[07:08,559] 卻沒有開任何 sandbox
[07:10,140] 講難聽一點就是在裸奔
[07:12,260] 第二層，高風險動作要人簽核
[07:14,739] 部署到 production，改資料庫結構
[07:17,279] 任何跟錢有關的操作，這種動作要設 checkpoint
[07:20,220] agent 走到哪裡必須停下來，等人類點頭
[07:23,459] 但簽核有一個陷阱
[07:24,559] AI 丟給你的是幾百行你看不懂的 code
[07:26,519] 你根本不知道自己在批准什麼
[07:28,160] 點同意只是一個儀式
[07:29,720] 所以教材提出了一個我認為非常棒的概念
[07:31,760] 叫 Vibe Diff
[07:32,959] 在你簽核之前
[07:34,119] 系統先把那段 code 翻譯回白話文
[07:36,459] 告訴你你當初的要求是什麼
[07:38,220] 以及它實際上打算做的事情是什麼
[07:40,619] 兩邊對得上你再批准

[07:42,359] 你批准的是你看得懂的東西
[07:44,579] 這個簽核才有意義
[07:45,980] 而不是一直沒有靈魂的按 enter
[07:48,380] 第三層，了解你會用到的套件
[07:50,660] 這裡有一個我認為所有人都該知道的攻擊手法
[07:53,440] 叫 slopsquatting
[07:54,839] AI 寫 code 的時候
[07:56,179] 常常會幻覺出根本不存在的套件名稱
[07:59,019] 而駭客早就發現這件事了
[08:01,079] 他們專門收集 AI 常見的幻覺名稱
[08:03,220] 搶先用這些名字上架真正的惡意套件
[08:05,660] 然後就在那邊等
[08:06,920] 等某個 agent 幻覺到這個名字
[08:08,920] 自動幫你裝進專案
[08:10,559] 防法很直接
[08:11,440] 套件只從公司核可的來源裝，版本釘死
[08:14,420] 上 production 之前用 CI 再掃一次
[08:16,880] 企業級的做法還有更狠的
[08:18,480] 像是專門養一隊 AI 天天攻擊自己家的 agent 的紅隊演練
[08:21,660] 不過那個離多數人比較遠，這邊就不展開
[08:24,480] 到這裡，講清楚讓它不用猜
[08:26,799] 設邊界讓它出錯也不至於出大事
[08:28,839] 但這兩個動作合起來
[08:30,500] 只保證了一件事，不出事
[08:32,520] 不出事跟做得好，是兩回事
[08:34,880] 因為你的 agent 可以安安穩穩的待在邊界裡
[08:37,479] 交出一個完全不能用的東西
[08:39,080] 所以第三個動作是把驗收做好
[08:41,500] 講驗收之前，要先講一個名詞叫做 observability
[08:44,979] 可觀測性
[08:46,159] 這個東西應該也相對好理解
[08:48,020] 簡單講就是要讓你的系統留下痕跡
[08:50,520] 因為你不能驗收你看不到的東西
[08:53,000] 傳統軟體的世界
[08:54,400] 系統回傳 200 OK 就代表成功
[08:56,739] 但 agent 的世界不是
[08:58,179] 它跟你說任務完成了
[08:59,799] 實際上它中間可能繞了二十圈冤枉路
[09:02,219] 或者掉進幻覺迴圈自己跟自己打架
[09:04,580] 最後硬湊出一個看起來像樣的結果
[09:06,539] 所以你需要把 agent 的整趟工作過程記錄下來
[09:10,219] 教材的做法有三層
[09:11,260] 第一層記錄整趟任務
[09:12,619] 第二層記錄它每一步在想什麼
[09:14,320] 第三層記錄它實際動了哪些工具
[09:16,460] 參數是什麼
[09:17,520] 有了這三層記錄
[09:18,679] agent 做出任何怪事
[09:19,900] 你都可以回放整個過程
[09:21,200] 找出它到底是在哪一步走歪的
[09:23,460] 還有一個一定要記錄的東西，就是花費
[09:25,619] 這次的教材中講到一個叫做 Denial of Wallet 的名詞
[09:28,440] 白話講就是駭客的目標不是偷你的資料
[09:30,840] 而是故意讓你的 agent 掉進一個無限迴圈
[09:33,759] 它每一圈都在呼叫要花錢的 API
[09:35,719] 就這樣安安靜靜的燒
[09:37,539] 駭客能不能賺到錢？不曉得
[09:39,479] 但你一定很痛苦

[09:41,059] 網路上什麼人都有
[09:42,260] 所以基本的防護還是要做好
[09:44,359] Day one 講 harness 六大件的時候我有說過
[09:47,260] 沒有 observability
[09:47,919] 你根本不知道 agent 是在做事
[09:49,500] 還是在偷偷浪費你的錢
[09:51,320] 看得到之後才是驗收本身
[09:53,419] 這裡要先劃一條線
[09:54,673] security 跟 evaluation 是兩件事
[09:57,059] security 告訴你 agent 沒有越界
[09:59,146] evaluation 則是告訴你
[10:00,250] 它在界線裡面做出來的東西
[10:01,780] 到底值不值得 ship 出去
[10:03,280] 一個 agent 完全可以通過所有安全檢查
[10:06,260] 然後交出一坨完全誤解你意圖的東西
[10:09,299] 那怎麼驗
[10:10,359] Day one 我們講過 tests 跟 evals 的分工
[10:13,679] tests 驗確定性的部分
[10:15,300] evals 驗非確定性的部分
[10:16,719] 這一集教材把這件事講得更深
[10:18,919] 傳統測試是二元的
[10:20,460] 這個 function 給這個輸入就該吐這個輸出
[10:22,719] 對就是對，錯就是錯
[10:24,460] 但 agent 的錯誤不長這樣
[10:26,059] 它可以一百個 unit test 全過
[10:27,679] 然後在真實任務裡選錯工具
[10:29,479] 或者把一句關鍵的回答改寫到意思跑掉
[10:32,359] 所以 evals 問的不是對錯
[10:33,880] 而是這一次的行為有沒有比基準好
[10:36,679] 它是用打分數，給容忍範圍的方式
[10:39,219] 抓出行為的飄移
[10:41,239] 但要注意，測試全綠不代表沒問題
[10:43,440] 因為 AI 會作弊
[10:44,719] 教材白紙黑字寫著
[10:46,500] agent 為了讓紅燈變綠燈
[10:48,340] 會直接把測試刪掉，或者把它 mock 掉
[10:50,960] 功能正確是地板，不是天花板
[10:53,580] 教材把 agent 的驗收拆成七個維度
[10:55,979] 內容太多我就不逐條唸
[10:57,479] 幫你收成兩組
[10:58,719] 更多細節我會放到 Patreon 的文章裡
[11:01,179] 第一組是使用者感受得到的
[11:03,039] 它有沒有做出你真正想要的東西
[11:05,200] code 能不能跑，畫面長得對不對
[11:07,820] 第二組是引擎蓋底下的
[11:09,660] 它解題的路徑合不合理
[11:10,919] 出錯之後是自我修復還是越修越爛
[11:13,340] 整趟燒了多少 token
[11:14,859] 你在意的通常是第一組
[11:16,000] 但第二組才是預測它下一次會不會出事的地方
[11:19,320] 最後給四個馬上可以用的驗收招式
[11:21,340] 這是我從教材裡挑出來最實戰的四個
[11:24,039] 招式一，把你自己的第一句話
[11:26,320] 存下來當作考題
[11:27,760] 你最初提的要求雖然很模糊，沒錯
[11:29,840] 但整個任務流程裡
[11:31,479] 最接近考題本質的東西
[11:33,039] 其實就是你開頭交代給它的那一兩句話

[11:35,659] 把這句話存下來
[11:36,859] 以後 AI 做的每一個步驟
[11:38,239] 都讓它回頭對著這句話重新打分數
[11:40,739] 看看自己有沒有偏題？
[11:42,260] 有沒有越做越偏離原始目標？
[11:44,440] 這招最聰明的地方在於
[11:46,119] 你根本不需要另外花時間想考題
[11:48,020] 因為你一開口的時候，考題就已經出好了
[11:51,039] 招式二，看成品，不看程式碼
[11:53,900] 如果今天 AI 做的是一個網頁
[11:55,679] 你驗收的時候
[11:56,739] 就直接去點開那個網頁看畫面
[11:58,880] 像是用手機打開發現跑版了
[12:01,299] 按鈕點下去沒反應
[12:02,299] 或是字體顏色太灰看不清楚
[12:04,580] 我跟你說
[12:05,239] 這些問題你就算盯著原始碼看一輩子也看不出來
[12:07,940] 但是網頁畫面一打開
[12:09,200] 你三秒鐘就能抓出毛病
[12:11,059] 既然使用者最後看到的是畫面
[12:13,260] 而不是底層的程式碼
[12:14,640] 那你驗收的時候
[12:15,659] 當然就應該直接驗畫面
[12:17,320] 招式三，看收斂程度
[12:19,059] 不要只看單一回合的對錯
[12:21,059] 交辦給 AI 一個任務
[12:22,700] 通常你們會來回溝通好幾輪
[12:24,760] 這裡的重點不是它在第四輪到底有沒有答對
[12:27,419] 而是你折騰到最後
[12:28,479] 到底有沒有拿到你要的東西？
[12:30,200] 如果它改了兩次就精準到位
[12:31,799] 那它就是個好員工
[12:33,119] 但如果它改了八次還是錯的
[12:35,440] 搞到最後你心累放棄，決定自己動手做
[12:38,559] 請注意，這種失敗的案子
[12:40,460] 千萬不要摸摸鼻子就算了
[12:42,400] 因為在所有的歷史紀錄裡
[12:44,140] 這種案子是最值得你回頭去研究的
[12:46,500] 這裡面一定藏著這個 AI 某種系統性的盲點或弱點
[12:50,359] 招式四，把你每一句罵它的話都存起來
[12:53,880] 你每次在對話框裡跟 AI 說不對
[12:56,380] 不是這樣，你搞錯了，這些話
[12:58,780] 其實就是一筆最棒的免費教材
[13:01,419] 因為它精準的標記出了
[13:03,200] AI 以為是對的，但你認為是錯的那個認知落差
[13:06,299] 把這些你糾正過的話累積起來
[13:08,799] 稍微分個類，你很快就會發現
[13:11,059] AI 犯的錯通常不是隨機的
[13:13,200] 而是會集中在少數幾種類型上
[13:15,359] 只要你找出那個錯誤集中的點
[13:16,880] 然後回過頭去修改你的 spec
[13:18,960] 修改你的死規則
[13:20,320] 這絕對比你自已憑空想像要怎麼出考題防範
[13:23,419] 還要快得多
[13:24,640] 這其實就是 Day one 說過的那件事
[13:27,019] agent 出包不要修完就走
[13:28,659] 要把答案寫回你的 harness
[13:30,840] 驗收標準一定要越客觀越好

[13:32,919] 幫我修到你覺得色彩夠和諧是一個壞標準
[13:35,659] 因為 AI 對和諧的理解跟你不一樣
[13:38,039] 這個指標達到多少就算完成才是好標準
[13:40,840] AI 能幫你自動化的範圍
[13:43,059] 取決於你能驗證的範圍
[13:44,979] 換句話說，你的驗證能力
[13:46,820] 就是你自動化能力的上限
[13:48,960] spec 有了，邊界有了，驗收也有了
[13:51,979] 機器該把的關都把了，那人呢
[13:54,559] Google 在教材裡誠實的承認了一件事
[13:56,820] AI 把寫 code 的瓶頸解決之後
[13:58,659] 生產力的瓶頸並沒有消失
[14:00,239] 而是移到了人身上
[14:01,700] 移到那個要 review 這一切的人身上
[14:04,400] Google 給出的案例是
[14:05,739] 一個 AI agent 可以在一個下午內
[14:07,640] 生出好幾個巨大的 PR
[14:09,380] PR 就是 pull request
[14:10,580] 意思是把寫好的 code 交給團隊審查
[14:13,380] 當 reviewer 看不完時
[14:14,739] 只好拆成一堆小 PR
[14:16,039] 結果變成俄羅斯套娃
[14:17,520] 一號 PR 要等二號，二號卡三號
[14:20,080] 三號的 reviewer 在另一個時區睡覺
[14:22,280] 整條線就這樣卡死
[14:23,739] 比塞車更嚴重的是人的狀態
[14:26,599] 重度使用 AI 的工作者
[14:27,960] 出現高度 burnout 的機率比不用的人高 45%
[14:31,239] 原因不難理解
[14:32,349] agent 每五分鐘就丟一個微小的決定給你
[14:34,979] 這行改不改，這個工具跑不跑
[14:37,239] 人被這種類似 micro management 轟炸久了
[14:39,840] 就會開始反射性的點同意，根本沒在看
[14:42,799] 這也是為什麼我在第二個動作特別強調 Vibe Diff
[14:45,619] 簽核的人如果看不懂自己在簽什麼
[14:47,580] 再多的關卡都只是走一個形式
[14:50,119] 教材給的解法中
[14:51,460] 其中一個叫 conditional L-G-T-M
[14:54,000] reviewer 先看架構，看完先批准
[14:56,539] 但這個批准是有條件的
[14:58,380] 條件是自動化測試全部通過
[15:00,440] 測試一綠，code 就自動合併
[15:02,080] 不用再等人醒來
[15:03,419] 跨時區那個十二小時的等待直接消失
[15:06,119] 另一個是把日常的把關
[15:07,400] 交給一個永遠不會累的 AI reviewer
[15:10,280] 讓它盯著你的 repo
[15:11,440] 每個 PR 進來自動審一輪
[15:13,440] 這種東西
[15:14,280] 從現成訂閱的服務
[15:15,979] 到自己寫一份 review skill 掛在 CI 上
[15:18,400] 再到全自建的客製 agent 都有
[15:20,500] 成本跟客製程度差很多
[15:22,739] 挑選原則就一句話
[15:24,179] 挑能抓到你在乎的問題的
[15:25,960] 最低的那一層就好
[15:27,000] 不要為了帥去 over engineering
[15:28,840] Day one 的影片結尾我引用了

[15:30,780] Google 這堂課程中的一句話
[15:32,520] Generation is solved, verification, judgment, and direction are the new craft
[15:37,140] 也就是產 code 的問題已經解決了
[15:39,619] 驗證，判斷，方向才是新的手藝
[15:42,200] 當時這只是一句宣言
[15:43,640] 五集走完，這句話有了完整的實作版本
[15:46,659] spec 是 direction，你把方向講清楚
[15:48,400] 邊界跟安全網是 judgment
[15:50,460] 你判斷哪些事它可以自己來
[15:52,159] 哪些事必須過你這一關
[15:54,200] observability 加 evals 是 verification
[15:56,260] 你看到它做了什麼，也驗得出做得好不好
[15:58,960] 這三件事
[16:00,059] 就是 Google 用五天的課程真正想教給我們的東西
[16:03,359] 如果你看完了這一系列的課程
[16:05,280] 但不知道從何下手，那我幫你想好了
[16:08,080] 第一，下一個功能開工之前，先寫 spec
[16:11,039] 十行也好
[16:12,059] 寫下你要什麼，為什麼，怎麼樣算做對
[16:14,659] 第二，現在就去研究
[16:15,760] 你的開發環境中要怎麼開一個 sandbox
[16:17,619] 不會的請直接問你的 AI，但別再裸奔了
[16:20,960] 第三，幫你最常跑的那個 workflow
[16:22,880] 寫下第一個 evaluation
[16:24,380] 定義清楚什麼叫好的結果
[16:26,159] 什麼叫壞的結果
[16:27,440] 這三件事都不難
[16:28,700] 但做了之後
[16:29,440] 你跟 AI 的協作就從碰運氣
[16:31,500] 變成一個可以持續改良的系統
[16:33,780] 這個系列的五份教材資訊量都很大
[16:35,760] 也有一些細節是我認為沒那麼重要的
[16:38,119] 但每一集我都有把完整內容
[16:39,679] 整理成一篇文章放在我的 Patreon
[16:41,719] 習慣用閱讀吸收的朋友可以在資訊欄找到連結
[16:44,559] 那以上就是今天的內容，希望對你們有所幫助
[16:47,000] 各位喜歡的話記得幫我按讚追蹤加分享
[16:49,020] 這是我持續創作下去的動力，我們下次見